Navigating Movie Taste with Sparse ALS and Genre Priors

Ahmed Amin [1]

Abstract

This study evaluates a movie recommender on all 32 million MovieLens ratings using a seeded 80/20 train/test split. The system combines alternating least squares (ALS) with movie-genre information. Ten-factor ALS attains a test RMSE of 0.78, compared with 1.06 for the training-mean baseline and 0.86 for a bias-only model. The genre prior yields a modest improvement when ratings are withheld for supported movies, but not for naturally unrated movies. A local application serves recommendations from the saved model, and independent checks reproduce the principal results. Code and supporting materials: GitHub repository.

1. Introduction

Movie services offer more films than an individual can browse. Ratings provide a sparse signal of preference because most viewers rate only a small fraction of a catalog. The resulting task is to estimate ratings for unseen films from observed user–item interactions. The evaluation therefore focuses on rating prediction error; lower error alone does not establish better ranking quality or user satisfaction.

Three approaches are evaluated: a training-set mean, a user–movie bias model, and alternating least squares (ALS) with latent user and movie vectors. The factor model captures interaction patterns shared across the dataset and is widely used in recommender systems (Koren et al., 2009). Genre information is evaluated as a prior, with particular attention to items with limited rating history. This design distinguishes standard prediction from cold-start settings for users or movies with sparse observations.

The experiments address four questions: the predictive value of personalization over mean and bias baselines; the accuracy–cost trade-off associated with model capacity; the effect of genre information under natural and controlled cold-start definitions; and the feasibility of serving local recommendations to a new user from a saved model. Root mean squared error (RMSE) measures rating prediction, while recommendation examples illustrate model use without constituting a ranking evaluation.

The saved model is also integrated into a local application that accepts ratings, excludes previously rated films, and presents recommendations. The application is a systems demonstration; clicks, ranked-list quality, and user satisfaction were not measured. MovieLens enables comparison on a large public dataset, but the findings apply to this dataset and split and should not be generalized to all viewers without further evaluation.

The study contributes a comparison of these methods using all 32 million MovieLens ratings and a fixed train/test split; analyses of model capacity, user activity, and natural versus controlled cold start; and a local application that fits a new-user profile without retraining movie parameters. Ten-factor ALS attains a test RMSE of 0.78, compared with 1.06 for the training-mean baseline and 0.86 for the bias model. Genre information modestly improves the controlled cold-start result but not the naturally unrated-item result. Because the analysis uses one random split, these estimates are comparative evidence rather than confirmation of gains on future data.

[1]African Institute for Mathematical Sciences (AIMS), South Africa. Correspondence to: Ahmed Amin <aamin@aims.ac.za>.



2. System Overview and Architecture

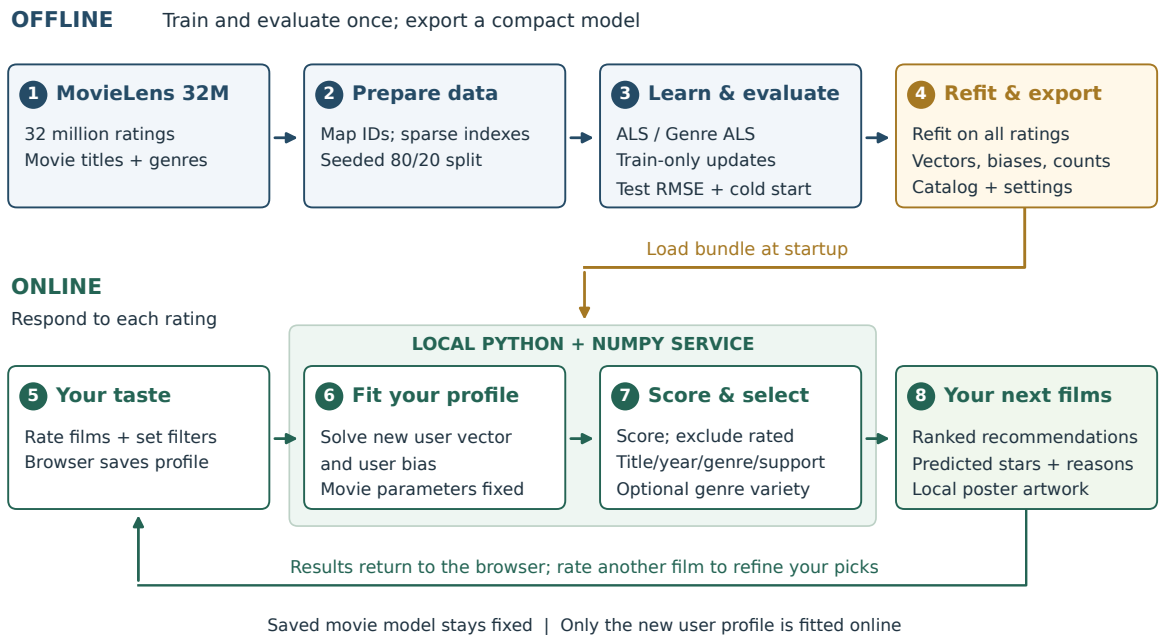


Figure 1. Full recommendation pipeline. The model is trained and exported once. At run time, a person's ratings update only that user's profile; the saved movie vectors score and filter recommendations.

Training is performed once on MovieLens, after which the movie parameters are saved. Figure 1 summarizes the serving pipeline: the local application loads the saved model, estimates a new user's profile from supplied ratings, and scores and filters unrated films. Movie parameters remain fixed during use. An optional genre setting modifies recommendation selection; its effect on recommendation quality and user satisfaction has not been evaluated.

3. Dataset and Exploratory Analysis

3.1. Data and Sparse Storage

MovieLens 32M contains 32,000,204 ratings from 200,948 users across 87,585 catalog movies (GroupLens Research, 2023; Harper & Konstan, 2015). Ratings range from 0.5 to 5 in half-star steps. The release selects users with at least 20 ratings. All observations and catalog movies are retained.

Table 1. Complete dataset and seeded 80/20 split.

Quantity	Count
All ratings	32,000,204
Users	200,948
Catalog movies	87,585
Training ratings	25,601,250
Test ratings	6,398,954
Movies without training ratings	7,319

The observed matrix is 0.18% dense; a dense float32 representation would need about 65.6 GiB. IDs map to contiguous indices. Chunked loading writes indices, ratings, and split membership to disk-backed arrays. Training entries are indexed in compressed sparse rows by both user and movie. Missing entries are unobserved preferences, not zero ratings.

The catalog includes 3,153 movies without any ratings and 7,319 without training ratings. Every user has training data, with a minimum of eight observations.

3.2. Distributions

Four stars is the most frequent rating (Figure 2). Activity and movie support have long tails (Figure 3), reflecting uneven feedback. Discrete frequency counts produce the horizontal tail bands; the plot alone does not establish a power law.

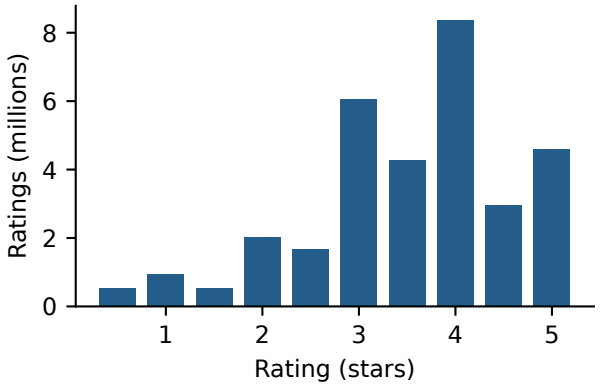


Figure 2. Rating distribution over all 32,000,204 observations.

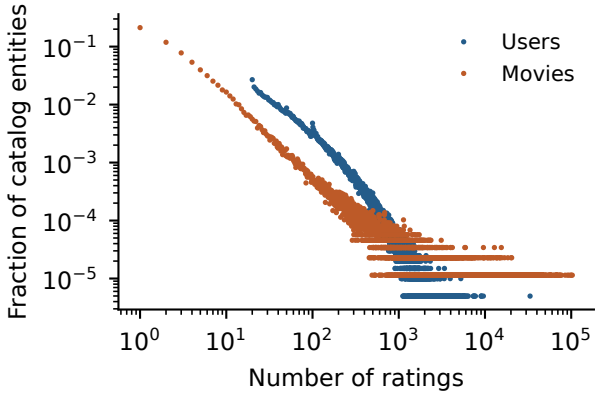


Figure 3. User and movie degrees. Zero-degree movies remain in the denominator but cannot appear on the log axis.

4. Methodology

The evaluated models are a training-set mean, a bias model, and ALS. User and movie biases represent deviations from the mean, while ALS additionally learns latent vectors whose inner product captures interaction structure. User and movie vectors are $\mathbf{u}_m, \mathbf{v}_n \in \mathbb{R}^K$, and their biases are $b_m^{(u)}, b_n^{(i)}$. $\Omega(m)$ denotes the movies rated by user m; $\Pi(n)$ denotes the users who rated movie n. Parameters λ, τ, and τ_b control the data-fit and shrinkage terms; the implementation names τ_b as `gam`.

4.1. Mean and Bias Baselines

The mean baseline predicts the training mean. The bias model predicts $\widehat{r}_{mn} = b_m^{(u)} + b_n^{(i)}$ and maximizes the log joint, up to constants:

$$\mathcal{L}_B = -\frac{\lambda}{2} \sum_{(m,n)\in T} (r_{mn} - \widehat{r}_{mn})^2 - \frac{\tau_b}{2} \left[\sum_m (b_m^{(u)})^2 + \sum_n (b_n^{(i)})^2 \right]. \tag{1}$$

Setting $\partial \mathcal{L}_B / \partial b_m^{(u)} = 0$ gives

$$b_m^{(u)} = \frac{\lambda \sum_{n\in\Omega(m)} (r_{mn} - b_n^{(i)})}{\lambda |\Omega(m)| + \tau_b}. \tag{2}$$

The movie update exchanges users and movies, using $\Pi(n)$. Empty rows receive zero bias. No global intercept is fitted ($\mu = 0$), so priors shrink biases toward zero.

4.2. Latent-Factor ALS

The prediction becomes

$$\widehat{r}_{mn} = \mathbf{u}_m^\mathsf{T} \mathbf{v}_n + b_m^{(u)} + b_n^{(i)}, \tag{3}$$

with log joint

$$\mathcal{L} = -\frac{\lambda}{2} \sum_{(m,n)\in T} (r_{mn} - \widehat{r}_{mn})^2 - \frac{\tau_b}{2} \left[\sum_m (b_m^{(u)})^2 + \sum_n (b_n^{(i)})^2 \right] - \frac{\tau}{2} \left[\sum_m \mathbf{u}_m^\mathsf{T} \mathbf{u}_m + \sum_n \mathbf{v}_n^\mathsf{T} \mathbf{v}_n \right]. \tag{4}$$

For fixed movies, update each user bias using Equation 2, subtracting the current interaction in its numerator. Then solve

$$\mathbf{A}_m = \lambda \sum_{n\in\Omega(m)} \mathbf{v}_n \mathbf{v}_n^\mathsf{T} + \tau \mathbf{I},$$
$$\mathbf{q}_m = \lambda \sum_{n\in\Omega(m)} (r_{mn} - b_m^{(u)} - b_n^{(i)}) \mathbf{v}_n,$$
$$\mathbf{A}_m \mathbf{u}_m = \mathbf{q}_m. \tag{5}$$

Movie updates are symmetric. Each sweep updates users, then movies. Since $\tau > 0$, the normal matrix is positive definite; the implementation solves the system without explicitly inverting it.

Exact block updates increase $\mathcal{L}$ despite joint nonconvexity. The code minimizes and plots $\mathcal{J} = -\mathcal{L}$, so its objective decreases. Factors start from Gaussian draws with standard deviation 0.1 and seed 42. Effective penalties are τ_b/λ and τ/λ; changing λ changes both.

4.3. Genre Prior

Let $\mathbf{x}_n \in \mathbb{R}^{19}$ assign $1/\sqrt{g_n}$ to each of movie n's g_n genres, or zero if none are listed. Rows of $\mathbf{F}$ are genre vectors; the prior mean is $\mathbf{F}^\mathsf{T}\mathbf{x}_n$. For movies W with training ratings, replace the movie penalty in Equation 4 to obtain

$$\begin{aligned}\mathcal{L}_G = &-\frac{\lambda}{2}\sum_T (r_{mn}-\widehat{r}_{mn})^2 \\ &-\frac{\tau_b}{2}\left[\sum_m (b_m^{(u)})^2 + \sum_n (b_n^{(i)})^2\right] \\ &-\frac{\tau}{2}\sum_m \|\mathbf{u}_m\|^2 - \frac{\tau}{2}\sum_{n\in W}\|\mathbf{v}_n - \mathbf{F}^\mathsf{T}\mathbf{x}_n\|^2 \\ &-\frac{\eta}{2}\|\mathbf{F}\|_F^2. \end{aligned} \tag{6}$$

The movie-vector equation gains $\tau\mathbf{F}^\mathsf{T}\mathbf{x}_n$ on its right side. With rows of $\mathbf{X}_W$ and $\mathbf{V}_W$ equal to $\mathbf{x}_n^\mathsf{T}$ and $\mathbf{v}_n^\mathsf{T}$, update genres through

$$(\tau\mathbf{X}_W^\mathsf{T}\mathbf{X}_W + \eta\mathbf{I})\mathbf{F} = \tau\mathbf{X}_W^\mathsf{T}\mathbf{V}_W. \tag{7}$$

Only warm movies enter this solve. A movie without training ratings receives $b_n^{(i)} = 0$ and $\mathbf{v}_n = \mathbf{F}^\mathsf{T}\mathbf{x}_n$. New users still require ratings or a fallback.

5. Experimental Setup and Implementation

5.1. Compiled Updates

Numba compiles the reference accumulations and linear solves. Users update independently with movies fixed, followed by the movie phase; parallel iterations write separate parameter rows (Numba Developers, 2026). Ratings use float32, while parameters and accumulated errors use float64. Batched evaluation avoids allocating a ratings-by-factors matrix. Per-sweep complexity is $O(|T|K^2 + (M+N)K^3)$.

The recorded Linux run used Python 3.13, NumPy 2.1, and Numba 0.61, with two Numba threads and one BLAS thread. Table 2 compares identical starting parameters after compilation warmup and verifies agreement with the reference. Compilation supplies the main speedup; parallel scheduling adds overhead on this bounded workload.

Table 2. One user-side update on 64,301 ratings, $K = 10$. Medians of three timings after warmup.

Method	Median (ms)	Speedup
Python reference	811.34	1.00×
Numba serial	14.85	54.64×
Numba parallel	15.34	52.91×

5.2. Evaluation Protocol

Seed 42 assigns each rating to training with probability 0.8 (Table 1). Updates use training data only. There is no validation set. Test RMSE is monitored, but factor models retain sweep 20 without early stopping or checkpoint selection.

The baseline fixes $K = 10$, $\lambda = 0.01$, $\tau_b = 0.001$, and $\tau = 0.1$. Comparisons change one setting: $K = 20$, $\tau = 0.01$, $\tau = 1$, or $\lambda = 0.1$. Genre ALS adds $\eta = 0.1$. Baseline settings also supply the genre and full-data refits; separate $K = 2$ models produce the embeddings.

For test observations S, evaluate unclipped predictions using

$$\mathrm{RMSE}(S) = \sqrt{\frac{1}{|S|}\sum_{(m,n)\in S}(r_{mn}-\widehat{r}_{mn})^2}. \tag{8}$$

Metrics are reported to two decimals; full-precision values remain in the result CSVs. Hyperparameters retain their exact values. The monitored test set is not an untouched final holdout.

5.3. Bias-History Correction

The bias run contains eight updates followed by twenty more with a restarted counter. Independent fitting reproduces all 28 updates. Figure 4 corrects only the cumulative-update axis; factor models still use 20 sweeps.

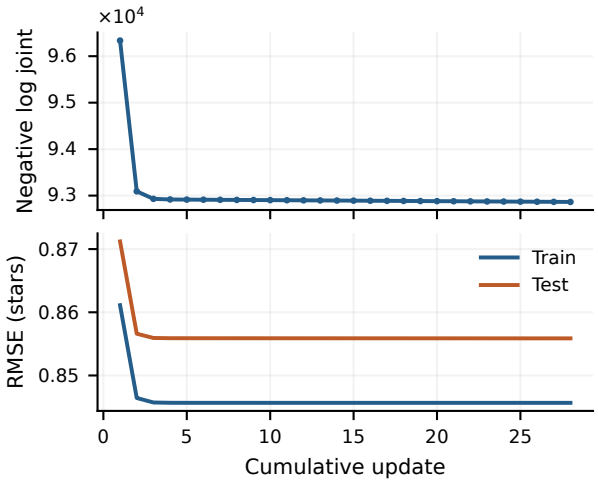


Figure 4. Bias fitting over 28 verified updates. The plotted objective is the negative log joint, $\mathcal{J}_B = -\mathcal{L}_B$.

6. Results and Discussion

6.1. Biases and Factors

Test RMSE falls from 1.06 for the training mean to 0.86 for biases and 0.78 for ten-factor ALS (Table 3). Interactions therefore improve substantially on user and movie offsets. Most improvement occurs in the first few sweeps, with little change near the fixed final checkpoint (Figure 5).

Table 3. Final test RMSE. The ALS variants tie at two-decimal precision; this does not imply identical predictions.

Model	Updates	Test RMSE
Training mean	–	1.06
Biases only	28	0.86
ALS, $K = 10$	20	0.78
ALS, $K = 20$	20	0.78
Genre ALS, $K = 10$	20	0.78

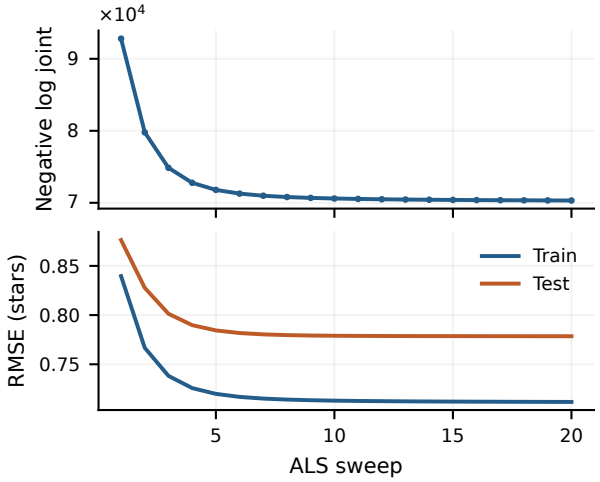


Figure 5. Ten-factor ALS: negative log joint decreases and train/test errors approach a plateau.

6.2. Capacity and Regularization

Twenty factors reduce training RMSE from 0.71 to 0.66, while both test errors round to 0.78. The small unrounded test gain costs 2.22 times more fitting time (Figure 6; Table 4).

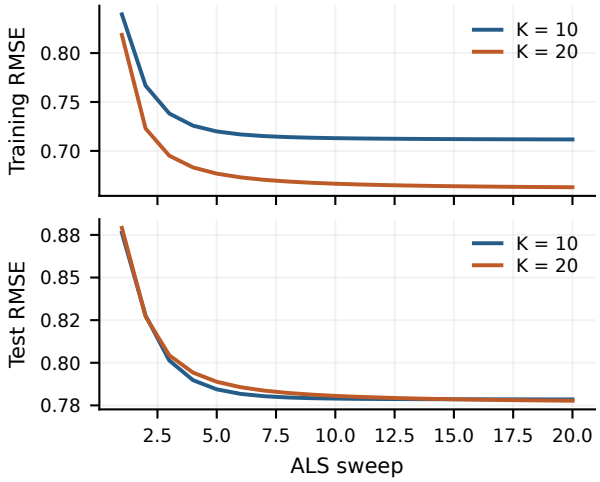


Figure 6. Training and test RMSE by factor count. Higher capacity lowers training error with little test improvement.

Table 4. ALS comparisons with $\tau_b = 0.001$ and 20 sweeps. Train and test columns report RMSE; times include training and evaluation.

Setting	K	λ	τ	Train	Test	Time (s)
Baseline	10	0.01	0.1	0.71	0.78	190.48
$K = 20$	20	0.01	0.1	0.66	0.78	423.75
$\tau = 0.01$	10	0.01	0.01	0.70	0.79	198.06
$\tau = 1$	10	0.01	1	0.77	0.80	184.43
$\lambda = 0.1$	10	0.1	0.1	0.70	0.79	184.50

Weakening τ to 0.01 or raising λ to 0.1 improves training fit but worsens test RMSE to 0.79. Their test errors reach minima at sweep 12, then rise while the training objective decreases: evidence of overfitting. Raising λ also weakens the effective bias penalty. Stronger $\tau = 1$ worsens both training and test fit.

Twenty factors help the most-active users but worsen both lower-activity groups (Table 5). The 100+ group contributes most test observations and drives the aggregate gain. Less-active users provide less evidence for larger vectors.

Table 5. Test RMSE by user training activity, using identical observations for both models.

Train ratings	Test rows	$K = 10$	$K = 20$
1–19	101,076	0.93	0.95
20–99	1,365,017	0.83	0.85
100+	4,932,861	0.76	0.75

6.3. Genre Prior and Cold Start

Genre ALS has training RMSE 0.71 and test RMSE 0.78. Its unrounded test improvement over ten-factor ALS is only 0.10%; one split does not establish significance. Test error reaches a shallow minimum at sweep 15, but the retained model remains sweep 20 (Figure 7).

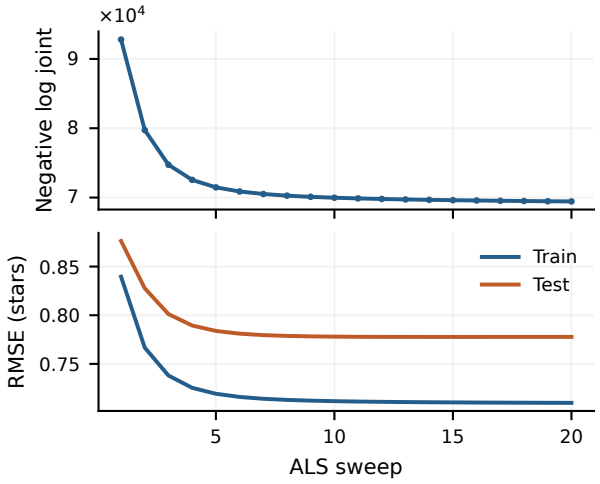


Figure 7. Genre ALS: negative log joint decreases; test RMSE has a shallow minimum at sweep 15.

Table 6. Warm-user test groups. The controlled study withholds training ratings for 1,063 supported movies.

Test group	Ratings	ALS	Genre ALS
Warm items	6,394,233	0.78	0.78
Natural cold items	4,721	1.03	1.04
Controlled cold items	329,121	0.99	0.97

For naturally cold items, genres worsen RMSE from 1.03 to 1.04 (Table 6). This group contains only 4,721 test ratings, so its effect on aggregate error is small.

The controlled study selects 5% of movies with at least 20 training ratings (seed 43), removes their training observations, and refits both models. Evaluation uses their test ratings from users who retain training data. Independent reconstruction verifies 1,063 movies and 329,121 test observations. Recorded RMSE improves from 0.99 to 0.97 with genres, a 2.22% reduction calculated before rounding.

Naturally unsupported and deliberately withheld movies are different populations. These results do not show that genres always improve cold start; the controlled study also lacks saved checkpoints for independent error recomputation.

6.4. Two-Dimensional Fits

Figures 8 and 9 show models trained directly with $K = 2$. Nearby Star Wars films and the Crime/Mystery genre pair illustrate learned relationships, but the axes have no intrinsic meaning because joint rotation or reflection preserves dot products. Both test RMSEs round to 0.81, above the main models.

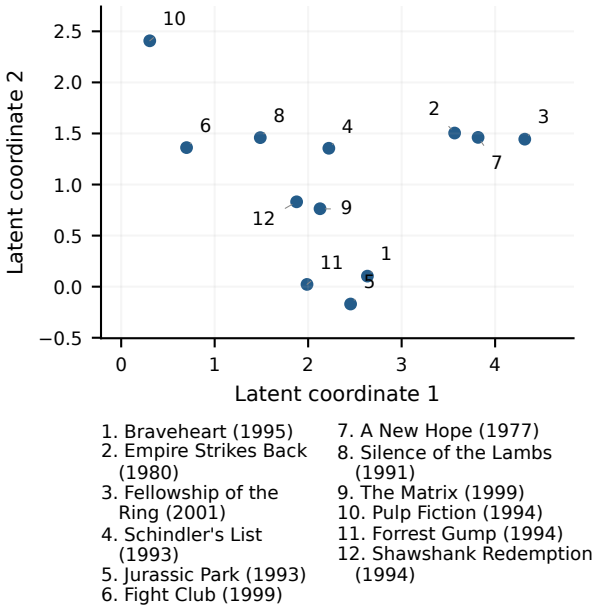


Figure 8. Direct two-factor movie fit. Numbers identify twelve frequently rated films; the key uses shortened titles.

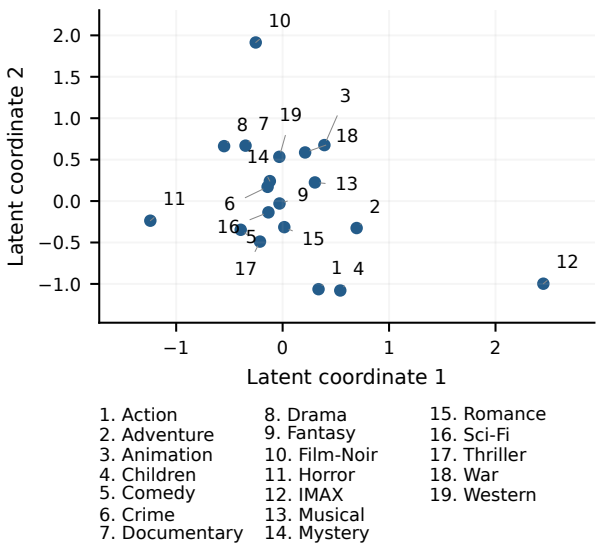


Figure 9. Direct two-factor genre fit. Labels are offset without moving the learned coordinates.

7. Recommendation Application and Verification

7.1. New-User Recommendations

After evaluation, both models are refitted on all ratings for 20 sweeps. Their training RMSEs round to 0.72; these are not test scores. For a new user m, let $\mathbf{D}$ have rows $[\mathbf{v}_n^\mathsf{T}, 1]$, $y_n = r_{mn} - b_n^{(i)}$, and $\boldsymbol{\theta} = [\mathbf{u}_m^\mathsf{T}, b_m^{(u)}]^\mathsf{T}$. With movie parameters fixed, solve

$$[\lambda \mathbf{D}^\mathsf{T}\mathbf{D} + \operatorname{diag}(\tau, \ldots, \tau, \tau_b)]\boldsymbol{\theta} = \lambda \mathbf{D}^\mathsf{T}\mathbf{y}. \tag{9}$$

Rank unrated movies by $\mathbf{u}_m^\mathsf{T}\mathbf{v}_n + b_n^{(i)}$; the user bias is constant. The practical's alternative weight $0.05b_n^{(i)}$ changes ranking, not predicted stars.

The other Lord of the Rings films lead the supported list (Table 7). Allowing single-rating movies instead produces raw predictions as high as 6.31, exposing weakly supported extrapolation. The dashboard filters support and clips displayed stars to 0.5–5; ranking uses raw scores.

Table 7. Genre-model recommendations after rating The Fellowship of the Ring five stars; support ≥ 100.

Recommended movie	Prediction	Support
The Return of the King (2003)	4.71	67,449
The Two Towers (2002)	4.67	67,463
A New Hope (1977)	4.30	85,010
The Empire Strikes Back (1980)	4.27	72,151
Return of the Jedi (1983)	4.16	67,496

7.2. Movie-Vector Norms

Table 8 lists the largest norms among movies with at least 100 ratings. Dumb & Dumber leads with norm 7.37 and rating standard deviation 1.20. Norms depend on factor scale and regularization, not just rating dispersion.

Table 8. Largest supported movie-vector norms. SD denotes rating standard deviation over all observations.

Movie	Norm	SD	Support
Dumb & Dumber (1994)	7.37	1.20	36,802
The Return of the King (2003)	6.54	1.01	67,449
The Two Towers (2002)	6.45	1.00	67,463
The Fellowship of the Ring (2001)	6.44	1.01	73,122
Deathly Hallows: Part 1 (2010)	6.28	1.02	20,969

7.3. Independent Checks

Independent checks verify data hashes and split membership and reproduce saved models' RMSEs, objectives, bias updates, activity groups, and recommendations. Controlled cold-start membership is verified, but its fitted checkpoints were not exported. Two-dimensional coordinates are available, but their checkpoints are also absent.

7.4. Limitations

A random rating split evaluates predictions for users and movies represented in the training data and does not establish performance on future ratings. The test set was monitored during development and is therefore not an untouched final holdout. Results cover one random split; neither multiple-seed uncertainty estimates, ranking benchmarks, nor user studies were obtained. Full-precision results and verification scripts accompany the report.

8. Conclusion

Sparse storage and compiled updates support ALS fitting on all 32 million ratings. Ten-factor ALS attains lower test RMSE than the mean and bias-only baselines. Increasing the factor count raises fitting time with little aggregate test improvement and degrades results for lower-activity users. Genre information improves the controlled cold-start estimate but not the estimate for naturally unrated movies. The local application demonstrates recommendation serving from saved movie parameters without retraining. Future work should evaluate temporally held-out ratings, ranking metrics, and user outcomes.

Acknowledgments

This project was developed as part of Prof. Ulrich Paquet's course. The author gratefully acknowledges his supervision and guidance and thanks the teaching assistant team for their support.

References

GroupLens Research. MovieLens 32M Dataset: README, 2023. URL https://files.grouplens.org/datasets/movielens/ml-32m-README.html. Release generated October 13, 2023; accessed October 27, 2026.

Harper, F. M. and Konstan, J. A. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015. doi: 10.1145/2827872.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, 2009. doi: 10.1109/MC.2009.263.

Numba Developers. Automatic parallelization with JIT, 2026. URL https://numba.readthedocs.io/en/stable/user/parallel.html. Accessed October 27, 2026.